MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Mini Project Report
of
Database Systems Lab (CSS 2236)

TITLE

SUBMITTED
BY

Parth Bagal 240957198 50 MnC-A
Aditya Ranka 240957140 41 MnC-A

School of Computer Engineering
Manipal Institute of Technology, Manipal.
April 2026

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

DEPARTMENT OF MATHEMATICS AND COMPUTING

Manipal
06/04/2026

CERTIFICATE

This is to certify that the project titled **Multi-User Analytical Strategy Management System Using Relational Databases** is a record of the bonafide work done by **Parth Bagal (240957198)** and **Aditya Ranka (240957140)** submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in Mathematics and Computing of Manipal Institute of Technology, Manipal, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Name and Signature of Examiners:

1. **Dr. Soumya Nandan Mishra, Assistant Professor, SCE.**

2. **Mr. Vinayak G Pai, Assistant Professor, SCE.**

TABLE OF CONTENTS

ABSTRACT

CHAPTER 1: INTRODUCTION

CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

CHAPTER 3: METHODOLOGY

CHAPTER 4: ER Diagram & Relational Tables along with sample data

CHAPTER 5: DDL commands & PL/SQL Procedures / Functions / Triggers

CHAPTER 6: RESULTS & SNAPSHOTS

CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK

CHAPTER 8: REFERENCES

Abstract

This project presents the design and implementation of a multi-user quantitative backtesting system built on a relational database architecture. The application enables users to configure analytical trading strategies, execute simulation runs against historical market data, and store structured results for historical comparison, portfolio optimization, and risk evaluation.

The core focus of the project is relational database design, entity relationships, transactional integrity, and efficient data retrieval. The system follows a hierarchical relational model consisting of eight entities: Users, Strategies, Strategy Parameters, Backtest Runs, Run Parameter Values, Daily Results, Performance Metrics, and Portfolio Weights. Each entity is connected via foreign key constraints with CASCADE semantics to maintain referential integrity.

Special attention is given to query optimization through composite indexing on foreign keys and time-series fields, efficient multi-table JOIN operations, and transaction management during bulk insert operations. The system implements triggers for data integrity enforcement, views for simplified reporting, and stored procedures for reusable query logic. The application is validated against 39 real backtest runs across four strategies and multiple Indian equity tickers.

Chapter 1: Introduction

1.1 Background

In modern data-driven environments, the ability to design, test, and evaluate analytical strategies systematically is critical across domains such as financial analysis, business intelligence, and quantitative research. As the volume and complexity of data grow, professionals and researchers increasingly rely on simulation-based systems to model strategies, assess their performance under varying conditions, and compare outcomes over time. However, managing such workflows without a structured, persistent data layer often leads to inconsistency, data loss, and an inability to draw meaningful comparisons across multiple runs or users.

Quantitative backtesting — the process of applying a trading or analytical strategy to historical data to evaluate its performance — is a foundational technique in financial engineering and systematic strategy research. When combined with portfolio optimization and risk modeling, it provides a rigorous framework for evaluating strategy robustness. The challenge, however, lies not just in the computation but in the structured storage, retrieval, and comparison of results across time and across multiple users.

1.2 Motivation

Existing backtesting tools are often standalone, single-user, and lack persistent database storage for simulation results. This makes it difficult to track strategy performance over time, compare parameter configurations, or share results across users. A well-designed relational database backend

addresses these limitations by providing structured, normalized storage with referential integrity, efficient querying, and multi-user support.

This project was motivated by the need to bridge the gap between quantitative strategy research and robust data management. By building a full-stack system centered on a relational database, the project demonstrates how database engineering principles can directly enable and enhance analytical research workflows.

1.3 Report Structure

The remainder of this report is organized as follows. Chapter 2 presents the problem statement and project objectives. Chapter 3 describes the methodology, including system architecture and the strategies implemented. Chapter 4 covers the ER diagram and relational schema with sample data. Chapter 5 presents the DDL commands and PL/SQL procedures, functions, and triggers. Chapter 6 discusses results and system snapshots. Chapter 7 concludes the report with limitations and future work, followed by references in Chapter 8.

CHAPTER 2: PROBLEM STATEMENT & OBJECTIVES

2.1 Problem Statement

Retail and semi-professional traders often backtest strategies incorrectly, ignoring transaction costs, failing to test statistical significance, overfitting parameters, and ignoring portfolio-level risk. This project addresses these issues by building a structured, database-backed framework that integrates strategy simulation, optimization theory, risk modeling, and statistical validation.

These issues collectively result in strategies that appear profitable in simulation but fail in practice. The absence of a structured, database-backed system is a root cause of many of these failures. This project addresses this gap by designing a relational database system that enforces structure, integrity, and consistency across all aspects of the backtesting workflow.

2.2 Project Objectives

Primary Objectives

• Develop a multi-user strategy backtesting engine backed by a normalized relational database
• Implement mean-variance portfolio optimization across strategy combinations
• Implement advanced risk modeling: VaR (Historical, Parametric, Monte Carlo), CVaR, and Drawdown Analysis
• Perform statistical robustness validation: stationarity tests, walk-forward validation, bootstrap Sharpe significance

Secondary Objectives

• Enable historical comparison of backtest runs with full parameter traceability
• Enforce data integrity via triggers, constraints, and referential integrity
• Optimize query performance using composite indexes and views

CHAPTER 3: METHODOLOGY

The system is divided into three distinct layers: a frontend interface layer, a backend application layer, and a SQL database layer. The tech stack is Python (Flask) for the backend, SQLite as the runtime database, and vanilla JavaScript/HTML/CSS for the frontend.

Layer	Technology	Responsibility
Frontend	HTML / CSS / JavaScript	User input, result display, charts
Backend	Python (Flask)	Business logic, API routing, DB connectivity
Database	SQLite (schema: PostgreSQL)	Normalized storage, query execution, triggers
Data Source	yfinance (Yahoo Finance)	Historical OHLCV data for Indian & US equities

Database Design Philosophy

The database design follows third normal form (3NF) to eliminate redundancy and ensure update consistency. Foreign key constraints enforce referential integrity across all parent–child relationships. Indexes are created on all foreign key columns and time-series fields to optimize JOIN performance and range queries. Transaction control is applied during bulk insert operations (e.g., inserting daily results) to ensure atomicity.

Database Connectivity

The Flask backend connects to SQLite via Python's built-in sqlite3 module. The connection is managed through a db.py module using a get_db() factory function. Foreign key enforcement is explicitly activated on every connection.

```
# backend/db.py import sqlite3 DB_PATH = "backtesting.db"  def get_db():     conn =
sqlite3.connect(DB_PATH)     conn.row_factory = sqlite3.Row     conn.execute("PRAGMA
foreign_keys = ON")  # Enable FK enforcement     return conn
```

All routes use parameterized queries to prevent SQL injection. Bulk inserts (e.g., writing all Daily_Results rows for a run) are wrapped in explicit transactions for atomicity.

CHAPTER 4: ER DIAGRAM & RELATIONAL TABLES

4.1 Entity-Relationship Overview

The schema consists of eight tables connected through a carefully designed set of foreign key relationships. The central fact table is Backtest_Runs, which connects Users, Strategies, and Market_Data. Performance_Metrics is a 1:1 extension of Backtest_Runs, and Daily_Results stores the time-series expansion of each run.

Relationship	Description
Users → Backtest_Runs	One user can execute many backtest runs (1:N)
Strategies → Strategy_Parameters	Each strategy owns a set of configurable parameters (1:N)
Backtest_Runs → Run_Parameter_Values	Each run stores the exact parameter values used (1:N, junction table)
Backtest_Runs → Daily_Results	Each run generates a daily time-series of signals and returns (1:N)
Backtest_Runs → Performance_Metrics	Each run has exactly one performance summary (1:1)
Backtest_Runs → Portfolio_Weights	A run can have multiple strategy allocation weights (1:N)

4.2 Relational Schema

The schema is implemented in two dialects: PostgreSQL (relationalSchema1.sql) for production, and SQLite (schema_sqlite.sql) for the runtime application. All eight tables are described below.

Users

Column	Type	Constraint	Description
user_id	SERIAL / INTEGER	PRIMARY KEY	Auto-incremented user identifier
username	VARCHAR(50)	UNIQUE NOT NULL	Unique login handle
email	VARCHAR(100)	UNIQUE NOT NULL	User email address
created_at	TIMESTAMP	DEFAULT NOW()	Account creation timestamp

Strategies

Column	Type	Constraint	Description
strategy_id	SERIAL / INTEGER	PRIMARY KEY	Auto-incremented strategy ID
strategy_name	VARCHAR(100)	UNIQUE NOT NULL	Strategy display name
description	TEXT	NULLABLE	Optional textual description
created_at	TIMESTAMP	DEFAULT NOW()	Creation timestamp

Strategy_Parameters

Column	Type	Constraint	Description
parameter_id	SERIAL / INTEGER	PRIMARY KEY	Auto-incremented parameter ID
strategy_id	INT	FK -> Strategies	Owning strategy
parameter_name	VARCHAR(50)	UNIQUE(sid,name)	e.g. short_window, z_threshold
data_type	VARCHAR(20)	NOT NULL	INT, FLOAT, etc.
default_value	VARCHAR(50)	NULLABLE	Default parameter value

Market_Data

Column	Type	Constraint	Description
data_id	SERIAL / INTEGER	PRIMARY KEY	Auto-incremented row ID
ticker_symbol	VARCHAR(20)	NOT NULL	e.g. RELIANCE.NS, INFY.BO
trade_date	DATE / TEXT	UNIQUE(ticker,date)	Trading date
open/high/low/close	DECIMAL(18,6)	NULLABLE	OHLC prices
volume	BIGINT	NULLABLE	Daily traded volume
adj_close	DECIMAL(18,6)	NULLABLE	Dividend-adjusted close

Backtest_Runs

Column	Type	Constraint	Description
run_id	SERIAL / INTEGER	PRIMARY KEY	Auto-incremented run ID
user_id	INT	FK -> Users CASCADE	Executing user
strategy_id	INT	FK -> Strategies CASCADE	Strategy used

ticker_symbol	VARCHAR(20)	NOT NULL	Asset backtested
start_date / end_date	DATE	CHECK end>start	Backtest date range
initial_capital	DECIMAL(18,2)	DEFAULT 100000	Starting capital in INR
executed_at	TIMESTAMP	DEFAULT NOW()	Run execution timestamp

Run_Parameter_Values

Column	Type	Constraint	Description
run_id	INT	PK, FK -> Backtest_Runs	Run reference
parameter_id	INT	PK, FK -> Strategy_Parameters	Parameter reference
parameter_value	VARCHAR(50)	NOT NULL	Actual value used in this run

Daily_Results

Column	Type	Constraint	Description
result_id	SERIAL / INTEGER	PRIMARY KEY	Auto-incremented row ID
run_id	INT	FK -> Backtest_Runs CASCADE	Parent run
trade_date	DATE / TEXT	UNIQUE(run_id, date)	Trading date
signal	VARCHAR(10)	CHECK IN (BUY,SELL,HOLD)	Trading signal generated
daily_return	DECIMAL(18,6)	NULLABLE	Net return after transaction costs
cumulative_return	DECIMAL(18,6)	NULLABLE	Running cumulative return
position_size	DECIMAL(18,6)	NULLABLE	Position fraction (0 or 1)

Performance_Metrics

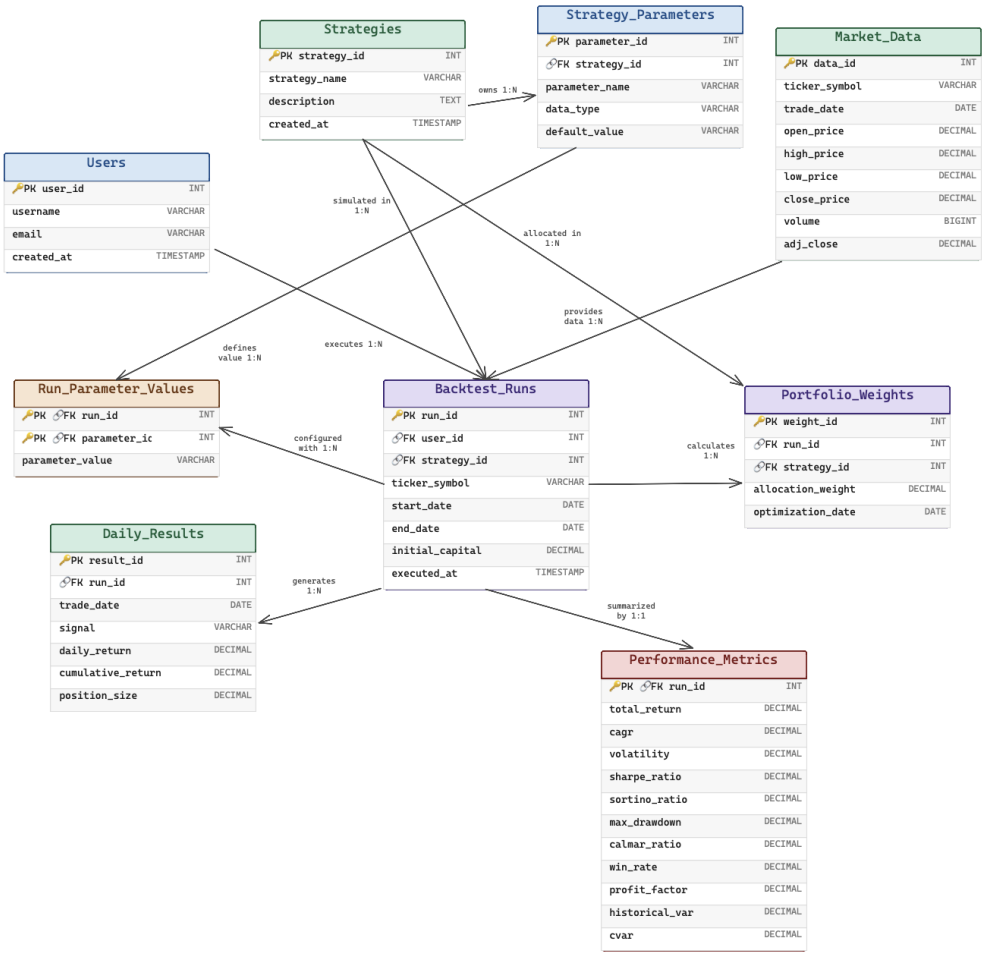
Column	Type	Constraint	Description
run_id	INT	PK = FK -> Backtest_Runs	1:1 relationship with run
total_return	DECIMAL	NULLABLE	Total strategy return

cagr	DECIMAL	NULLABLE	Compound annual growth rate
volatility	DECIMAL	NULLABLE	Annualized return volatility
sharpe_ratio	DECIMAL	NULLABLE	Risk-adjusted return
sortino_ratio	DECIMAL	NULLABLE	Downside risk-adjusted return
max_drawdown	DECIMAL	NULLABLE	Worst peak-to-trough drop
calmar_ratio	DECIMAL	NULLABLE	CAGR / Max Drawdown
win_rate	DECIMAL	CHECK 0<=x<=1	Fraction of profitable days
profit_factor	DECIMAL	NULLABLE	Gross Profit / Gross Loss
historical_var	DECIMAL	NULLABLE	Historical Value at Risk (5%)
cvar	DECIMAL	NULLABLE	Conditional VaR (5%)

Portfolio_Weights

Column	Type	Constraint	Description
weight_id	SERIAL / INTEGER	PRIMARY KEY	Auto-incremented ID
run_id	INT	FK -> Backtest_Runs CASCADE	Parent run
strategy_id	INT	FK -> Strategies CASCADE	Strategy being allocated
allocation_weight	DECIMAL(10,6)	CHECK 0<=x<=1	Fraction allocated to strategy
optimization_date	DATE / TEXT	NOT NULL	Date of optimization

4.3 ER Diagram



CHAPTER 5: DDL COMMANDS & PL/SQL PROCEDURES / FUNCTIONS / TRIGGERS

5.1 DDL Commands

The following Data Definition Language (DDL) commands define the complete relational schema. Both PostgreSQL (SERIAL, DECIMAL) and SQLite (INTEGER AUTOINCREMENT, REAL) dialects are used in this project; the PostgreSQL version is shown below for its PL/SQL compatibility.

```
-- USERS TABLE
CREATE TABLE Users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- STRATEGIES TABLE
```

```
CREATE TABLE Strategies (
    strategy_id SERIAL PRIMARY KEY,
    strategy_name VARCHAR(100) UNIQUE NOT NULL,
    description TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- STRATEGY PARAMETERS TABLE
CREATE TABLE Strategy_Parameters (
    parameter_id SERIAL PRIMARY KEY,
    strategy_id INT NOT NULL REFERENCES Strategies(strategy_id) ON DELETE CASCADE,
    parameter_name VARCHAR(50) NOT NULL,
    data_type VARCHAR(20) NOT NULL,
    default_value VARCHAR(50),
    UNIQUE(strategy_id, parameter_name)
);
CREATE INDEX idx_strategy_params_strategy ON Strategy_Parameters(strategy_id);

-- MARKET DATA TABLE
CREATE TABLE Market_Data (
    data_id SERIAL PRIMARY KEY,
    ticker_symbol VARCHAR(20) NOT NULL,
    trade_date DATE NOT NULL,
    open_price DECIMAL(18,6), high_price DECIMAL(18,6),
    low_price DECIMAL(18,6), close_price DECIMAL(18,6),
    volume BIGINT, adj_close DECIMAL(18,6),
    UNIQUE(ticker_symbol, trade_date)
);
CREATE INDEX idx_market_data_date ON Market_Data(trade_date);
CREATE INDEX idx_market_data_ticker ON Market_Data(ticker_symbol);

-- BACKTEST RUNS TABLE
CREATE TABLE Backtest_Runs (
    run_id SERIAL PRIMARY KEY,
    user_id INT NOT NULL REFERENCES Users(user_id) ON DELETE CASCADE,
    strategy_id INT NOT NULL REFERENCES Strategies(strategy_id) ON DELETE CASCADE,
    ticker_symbol VARCHAR(20) NOT NULL,
    start_date DATE NOT NULL, end_date DATE NOT NULL,
    initial_capital DECIMAL(18,2) DEFAULT 100000.00,
    executed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CHECK (end_date > start_date)
);
CREATE INDEX idx_backtest_runs_user ON Backtest_Runs(user_id);
CREATE INDEX idx_backtest_runs_strategy ON Backtest_Runs(strategy_id);

-- RUN PARAMETER VALUES TABLE
CREATE TABLE Run_Parameter_Values (
    run_id INT NOT NULL REFERENCES Backtest_Runs(run_id) ON DELETE CASCADE,
```

```
    parameter_id INT NOT NULL REFERENCES Strategy_Parameters(parameter_id) ON
DELETE CASCADE,
    parameter_value VARCHAR(50) NOT NULL,
    PRIMARY KEY (run_id, parameter_id)
);

-- DAILY RESULTS TABLE
CREATE TABLE Daily_Results (
    result_id SERIAL PRIMARY KEY,
    run_id INT NOT NULL REFERENCES Backtest_Runs(run_id) ON DELETE CASCADE,
    trade_date DATE NOT NULL,
    signal VARCHAR(10),
    daily_return DECIMAL(18,6), cumulative_return DECIMAL(18,6),
    position_size DECIMAL(18,6),
    UNIQUE(run_id, trade_date)
);
CREATE INDEX idx_daily_results_run ON Daily_Results(run_id);
CREATE INDEX idx_daily_results_run_date ON Daily_Results(run_id, trade_date);

-- PERFORMANCE METRICS TABLE
CREATE TABLE Performance_Metrics (
    run_id INT PRIMARY KEY REFERENCES Backtest_Runs(run_id) ON DELETE
CASCADE,
    total_return DECIMAL(18,6), cagr DECIMAL(10,6), volatility DECIMAL(10,6),
    sharpe_ratio DECIMAL(10,6), sortino_ratio DECIMAL(10,6),
    max_drawdown DECIMAL(10,6), calmar_ratio DECIMAL(10,6),
    win_rate DECIMAL(10,6), profit_factor DECIMAL(10,6),
    historical_var DECIMAL(10,6), cvar DECIMAL(10,6),
    CHECK (win_rate >= 0 AND win_rate <= 1)
);

-- PORTFOLIO WEIGHTS TABLE
CREATE TABLE Portfolio_Weights (
    weight_id SERIAL PRIMARY KEY,
    run_id INT NOT NULL REFERENCES Backtest_Runs(run_id) ON DELETE CASCADE,
    strategy_id INT NOT NULL REFERENCES Strategies(strategy_id) ON DELETE CASCADE,
    allocation_weight DECIMAL(10,6) NOT NULL,
    optimization_date DATE NOT NULL,
    UNIQUE(run_id, strategy_id),
    CHECK (allocation_weight >= 0 AND allocation_weight <= 1)
);
CREATE INDEX idx_portfolio_weights_run ON Portfolio_Weights(run_id);
CREATE INDEX idx_portfolio_weights_strategy ON Portfolio_Weights(strategy_id);
```

SQLite Implementation

```
CREATE TRIGGER trg_validate_portfolio_weights AFTER INSERT ON Portfolio_Weights FOR EACH ROW BEGIN
SELECT RAISE(ABORT, 'Portfolio weights for this run and date exceed 1.0')     WHERE (         SELECT SUM(allocation_weight)
```

```
FROM Portfolio_Weights WHERE run_id = NEW.run_id AND optimization_date = NEW.optimization_date ) > 1.0; END;
```

PostgreSQL Implementation

```
CREATE OR REPLACE FUNCTION fn_validate_portfolio_weights() RETURNS TRIGGER AS $$ DECLARE total_weight DECIMAL; BEGIN SELECT SUM(allocation_weight) INTO total_weight FROM Portfolio_Weights WHERE run_id = NEW.run_id AND optimization_date = NEW.optimization_date; IF total_weight > 1.0 THEN RAISE EXCEPTION 'Portfolio weights exceed 1.0'; END IF; RETURN NEW; END; $$ LANGUAGE plpgsql; CREATE TRIGGER trg_validate_portfolio_weights AFTER INSERT OR UPDATE ON Portfolio_Weights FOR EACH ROW EXECUTE FUNCTION fn_validate_portfolio_weights();
```

Portfolio Allocation History View

```
CREATE VIEW v_portfolio_allocation_history AS SELECT pw.run_id, pw.optimization_date, s.strategy_name, pw.allocation_weight FROM Portfolio_Weights pw JOIN Strategies s ON pw.strategy_id = s.strategy_id JOIN Backtest_Runs br ON pw.run_id = br.run_id ORDER BY pw.run_id, pw.optimization_date, s.strategy_name;
```

Stored Procedure: Run Summary

```
CREATE OR REPLACE PROCEDURE sp_get_run_summary(IN p_run_id INT) LANGUAGE plpgsql AS $$ BEGIN SELECT br.ticker_symbol, br.start_date, br.end_date, pm.sharpe_ratio, pm.max_drawdown, pm.total_return FROM Backtest_Runs br JOIN Performance_Metrics pm ON br.run_id = pm.run_id WHERE br.run_id = p_run_id; END; $$;
```

CHAPTER 6: RESULTS & SNAPSHOTS

6.1 Basic Query: All Backtest Runs

A simple SELECT query retrieving all 39 backtest runs stored in the database, showing run_id, user_id, strategy_id, ticker_symbol, date range, capital, and execution timestamp.

```
SELECT * FROM Backtest_Runs;
```

```
==============================================================================
1. All rows from Backtest_Runs
==============================================================================
run_id | user_id | strategy_id | ticker_symbol | start_date | end_date   | initial_capital | executed_at
-----------------------------------------------------------------------------------------------------------------
1      | 1       | 2           | AAPL          | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 11:56:20
2      | 1       | 3           | AAPL          | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 11:56:27
3      | 1       | 1           | AAPL          | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 11:56:37
4      | 1       | 4           | AAPL          | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 11:56:44
5      | 1       | 1           | RELIANCE.NS   | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 11:58:25
6      | 1       | 2           | RELIANCE.NS   | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 11:58:34
7      | 1       | 3           | RELIANCE.NS   | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 11:58:40
8      | 1       | 4           | RELIANCE.NS   | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 11:58:45
9      | 1       | 1           | RELIANCE.NS   | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 12:09:55
10     | 1       | 1           | TCS.NS        | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 12:10:18
11     | 1       | 1           | HDFCBANK.NS   | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 12:11:14
12     | 1       | 1           | HDFCBANK.NS   | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:12:18
13     | 1       | 1           | INFY.BO       | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:12:31
14     | 1       | 2           | INFY.BO       | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:12:37
15     | 1       | 3           | INFY.BO       | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:12:43
16     | 1       | 4           | INFY.BO       | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:12:47
17     | 1       | 1           | INFY.BO       | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:12:50
18     | 1       | 1           | ICICIBANK.NS  | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:12:59
19     | 1       | 2           | ICICIBANK.NS  | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:13:03
20     | 1       | 3           | ICICIBANK.NS  | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:13:07
21     | 1       | 4           | ICICIBANK.NS  | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:13:11
22     | 1       | 4           | RELIANCE.NS   | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:13:39
23     | 1       | 3           | RELIANCE.NS   | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:13:43
24     | 1       | 2           | RELIANCE.NS   | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:13:46
25     | 1       | 1           | RELIANCE.NS   | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 12:13:50
26     | 1       | 1           | INFY.BO       | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 16:12:27
27     | 1       | 1           | RELIANCE.NS   | 2025-03-20 | 2026-03-20 | 100000.0        | 2026-03-26 16:13:15
28     | 1       | 4           | ICICIBANK.NS  | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 16:15:22
29     | 1       | 2           | ICICIBANK.NS  | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 16:15:34
30     | 1       | 3           | ICICIBANK.NS  | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 16:15:42
31     | 1       | 1           | RELIANCE.NS   | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-03-26 16:16:42
32     | 1       | 1           | RELIANCE.NS   | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-04-05 15:29:17
33     | 1       | 1           | RELIANCE.NS   | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-04-05 15:45:05
34     | 1       | 2           | RELIANCE.NS   | 2023-01-01 | 2024-12-31 | 100000.0        | 2026-04-05 16:01:57
35     | 1       | 1           | RELIANCE.NS   | 2025-03-31 | 2026-03-31 | 100000.0        | 2026-04-05 16:02:35
36     | 1       | 3           | RELIANCE.NS   | 2025-03-31 | 2026-03-31 | 100000.0        | 2026-04-05 16:02:44
37     | 1       | 4           | RELIANCE.NS   | 2025-03-31 | 2026-03-31 | 100000.0        | 2026-04-05 16:02:48
38     | 1       | 1           | RELIANCE.NS   | 2025-03-31 | 2026-03-31 | 100000.0        | 2026-04-05 16:02:51
39     | 1       | 1           | INFY.BO       | 2025-03-31 | 2026-03-31 | 100000.0        | 2026-04-05 16:03:00
```

Figure 2: All 39 backtest runs across multiple tickers and strategies

6.2 Basic Query: All Performance Metrics

Retrieves the complete performance metrics table showing Sharpe ratio, Sortino ratio, max drawdown, volatility, CAGR, win rate, profit factor, VaR, and CVaR for all 39 runs.

```
SELECT * FROM Performance_Metrics;
```

```
==============================================================================
2. All rows from Performance_Metrics
==============================================================================
```

run_id	total_return	cagr	volatility	sharpe_ratio	sortino_ratio	max_drawdown	calmar_ratio	win_rate	profit_factor	historical_var	cvar
1	-0.042408	-0.021561	0.101885	-0.359463	-0.344026	-0.136497	-0.157956	0.360406	0.953533	-0.009748	-0.016924
2	-0.134222	-0.069929	0.117702	-0.726938	-0.6554	-0.147842	-0.473	0.35567	0.85077	-0.012218	-0.019945
3	0.177828	0.08581	0.199897	0.411545	0.585379	-0.242002	0.354583	0.523385	1.096683	-0.019172	-0.028125
4	-0.025295	-0.012804	0.212177	-0.04964	-0.082502	-0.347159	-0.036882	0.503132	1.007752	-0.021372	-0.0271
5	0.41836	0.19691	0.192721	0.924911	1.360297	-0.126956	1.551006	0.515909	1.211229	-0.017049	-0.025758
6	-0.096822	-0.051025	0.109061	-0.609735	-0.644193	-0.228697	-0.22311	0.318681	0.871422	-0.01014	-0.016203
7	0.139916	0.069668	0.106715	0.496301	0.541416	-0.102154	0.681985	0.406593	1.257187	-0.00826	-0.013993
8	-0.337818	-0.191032	0.203329	-1.038589	-1.353775	-0.444212	-0.430048	0.480851	0.845658	-0.020176	-0.031276
9	0.41836	0.19691	0.192721	0.924911	1.360297	-0.126956	1.551006	0.515909	1.211229	-0.017049	-0.025758
10	-0.081421	-0.042737	0.186253	-0.249064	-0.351209	-0.195635	-0.218452	0.486364	0.974038	-0.018827	-0.026844
11	-0.28384	-0.157764	0.188674	-0.920856	-1.129013	-0.348331	-0.452916	0.472727	0.855539	-0.017074	-0.029493
12	0.119458	0.121497	0.134974	0.768403	1.168546	-0.132867	0.914425	0.548223	1.19603	-0.012667	-0.01684
13	0.207659	0.21228	0.218423	0.899092	1.102259	-0.11481	1.848972	0.532995	1.217313	-0.018457	-0.031895
14	-0.058895	-0.06005	0.113101	-0.667454	-0.440523	-0.147862	-0.406124	0.43038	0.841743	-0.010368	-0.02016
15	-0.215498	-0.219343	0.153589	-1.663648	-1.0894	-0.261275	-0.839508	0.382716	0.577667	-0.013853	-0.028197
16	-0.03053	-0.031138	0.243318	-0.091294	-0.127378	-0.301091	-0.103418	0.471366	0.998264	-0.021684	-0.034491
17	0.207659	0.21228	0.218423	0.899092	1.102259	-0.11481	1.848972	0.532995	1.217313	-0.018457	-0.031895
18	-0.172882	-0.175411	0.144197	-1.403867	-1.870633	-0.253009	-0.693297	0.446701	0.786435	-0.0144	-0.021734
19	-0.00243	-0.002469	0.087284	-0.213985	-0.168845	-0.087667	-0.02816	0.333333	1.005321	-0.006341	-0.014264
20	0.035908	0.036498	0.090254	0.220566	0.145279	-0.08236	0.443155	0.383333	1.201819	-0.00197	-0.013194
21	-0.351823	-0.356341	0.162792	-2.745726	-3.730427	-0.418069	-0.852348	0.444934	0.620714	-0.018869	-0.025234
22	-0.072376	-0.073499	0.187495	-0.420023	-0.543939	-0.143141	-0.513474	0.502203	0.944706	-0.01677	-0.028956
23	-0.061632	-0.062595	0.106611	-0.740584	-0.584924	-0.11637	-0.537893	0.4	0.842319	-0.011434	-0.019058
24	-0.162695	-0.165089	0.092097	-2.129448	-1.434926	-0.188666	-0.875033	0.352113	0.51051	-0.010899	-0.01936
25	-0.033943	-0.034481	0.161158	-0.261281	-0.324527	-0.147478	-0.233804	0.472081	0.97409	-0.013694	-0.024948
26	0.207659	0.21228	0.218423	0.899092	1.102259	-0.11481	1.848972	0.532995	1.217313	-0.018457	-0.031895
27	-0.033943	-0.034481	0.161158	-0.261281	-0.324527	-0.147478	-0.233804	0.472081	0.97409	-0.013694	-0.024948
28	-0.425714	-0.248164	0.181662	-1.588197	-2.038236	-0.432127	-0.574284	0.468085	0.76494	-0.01978	-0.028304
29	-0.042426	-0.022049	0.103372	-0.357973	-0.300611	-0.130088	-0.169491	0.365269	0.940298	-0.006089	-0.015911
30	0.07957	0.040161	0.083908	0.272828	0.226385	-0.0986	0.40731	0.41875	1.17309	-0.006465	-0.013424
31	0.41836	0.19691	0.192721	0.924911	1.360297	-0.126956	1.551006	0.515909	1.211229	-0.017049	-0.025758
32	0.41836	0.19691	0.192721	0.924911	1.360297	-0.126956	1.551006	0.515909	1.211229	-0.017049	-0.025758
33	0.41836	0.19691	0.192721	0.924911	1.360297	-0.126956	1.551006	0.515909	1.211229	-0.017049	-0.025758
34	-0.096822	-0.051025	0.109061	-0.609735	-0.644193	-0.228697	-0.22311	0.318681	0.871422	-0.01014	-0.016203
35	-0.003966	-0.004046	0.168725	-0.058431	-0.074674	-0.147478	-0.027435	0.469388	1.011789	-0.013991	-0.025537
36	-0.061632	-0.06284	0.106827	-0.741319	-0.586692	-0.11637	-0.540001	0.4	0.842319	-0.011462	-0.019058
37	-0.046828	-0.047753	0.185249	-0.279869	-0.402087	-0.149114	-0.320248	0.49115	0.969457	-0.016869	-0.026554
38	-0.003966	-0.004046	0.168725	-0.058431	-0.074674	-0.147478	-0.027435	0.469388	1.011789	-0.013991	-0.025537
39	0.126339	0.129612	0.219428	0.573921	0.701908	-0.11481	1.128931	0.520408	1.141679	-0.019947	-0.032608

Figure 3: Performance metrics for all backtest runs

6.3 Complex Query: Sharpe Ratio Ranking

A multi-table JOIN across Backtest_Runs, Strategies, and Performance_Metrics, ordered by Sharpe ratio descending. This demonstrates efficient use of FK relationships and composite indexing.

```
SELECT br.ticker_symbol, s.strategy_name, pm.sharpe_ratio FROM Backtest_Runs br JOIN Strategies s ON br.strategy_id = s.strategy_id JOIN Performance_Metrics pm ON br.run_id = pm.run_id ORDER BY pm.sharpe_ratio DESC;
```

```
================================================================================
4. Sharpe ratio ranking
================================================================================
ticker_symbol | strategy_name            | sharpe_ratio
-----------------------------------------------------------
RELIANCE.NS   | Moving Average Crossover | 0.924911
RELIANCE.NS   | Moving Average Crossover | 0.924911
RELIANCE.NS   | Moving Average Crossover | 0.924911
RELIANCE.NS   | Moving Average Crossover | 0.924911
RELIANCE.NS   | Moving Average Crossover | 0.924911
INFY.BO       | Moving Average Crossover | 0.899092
INFY.BO       | Moving Average Crossover | 0.899092
INFY.BO       | Moving Average Crossover | 0.899092
HDFCBANK.NS   | Moving Average Crossover | 0.768403
INFY.BO       | Moving Average Crossover | 0.573921
RELIANCE.NS   | RSI Reversal             | 0.496301
AAPL          | Moving Average Crossover | 0.411545
ICICIBANK.NS  | RSI Reversal             | 0.272828
ICICIBANK.NS  | RSI Reversal             | 0.220566
AAPL          | Momentum                 | -0.04964
RELIANCE.NS   | Moving Average Crossover | -0.058431
RELIANCE.NS   | Moving Average Crossover | -0.058431
INFY.BO       | Momentum                 | -0.091294
ICICIBANK.NS  | Mean Reversion           | -0.213985
TCS.NS        | Moving Average Crossover | -0.249064
RELIANCE.NS   | Moving Average Crossover | -0.261281
RELIANCE.NS   | Moving Average Crossover | -0.261281
RELIANCE.NS   | Momentum                 | -0.279869
ICICIBANK.NS  | Mean Reversion           | -0.357973
AAPL          | Mean Reversion           | -0.359463
RELIANCE.NS   | Momentum                 | -0.420023
RELIANCE.NS   | Mean Reversion           | -0.609735
RELIANCE.NS   | Mean Reversion           | -0.609735
INFY.BO       | Mean Reversion           | -0.667454
AAPL          | RSI Reversal             | -0.726938
RELIANCE.NS   | RSI Reversal             | -0.740584
RELIANCE.NS   | RSI Reversal             | -0.741319
HDFCBANK.NS   | Moving Average Crossover | -0.920856
RELIANCE.NS   | Momentum                 | -1.038589
ICICIBANK.NS  | Moving Average Crossover | -1.403867
ICICIBANK.NS  | Momentum                 | -1.588197
INFY.BO       | RSI Reversal             | -1.663648
RELIANCE.NS   | Mean Reversion           | -2.129448
ICICIBANK.NS  | Momentum                 | -2.745726

================================================================================
5. Average sharpe and drawdown per strategy with run count
================================================================================
strategy_name            | run_count | avg_sharpe          | avg_drawdown
--------------------------------------------------------------------------------
Mean Reversion           | 7         | -0.7068275714285714 | -0.16402485714285714
Momentum                 | 7         | -0.8876197142857143 | -0.31927328571428576
Moving Average Crossover | 18        | 0.32569383333333335 | -0.15865422222222222
RSI Reversal             | 7         | -0.4118277142857143 | -0.13213871428571428
```

Figure 4: Sharpe ratio ranking — Moving Average Crossover on RELIANCE.NS leads with Sharpe 0.9249

6.4 Complex Query: Aggregate Metrics per Strategy

A GROUP BY query computing average Sharpe ratio, average max drawdown, and total run count per strategy. This demonstrates aggregation across joined tables.

```
SELECT s.strategy_name,        COUNT(*) AS run_count,        AVG(pm.sharpe_ratio) AS avg_sharpe,        AVG(pm.max_drawdown) AS avg_drawdown FROM Strategies s JOIN Backtest_Runs br ON s.strategy_id = br.strategy_id JOIN Performance_Metrics pm ON br.run_id = pm.run_id GROUP BY s.strategy_name;
```

```
================================================================================
5. Average sharpe and drawdown per strategy with run count
================================================================================
strategy_name            | run_count | avg_sharpe          | avg_drawdown
--------------------------------------------------------------------------------
Mean Reversion           | 7         | -0.7068275714285714 | -0.16402485714285714
Momentum                 | 7         | -0.8876197142857143 | -0.31927328571428576
Moving Average Crossover | 18        | 0.32569383333333335 | -0.15865422222222222
RSI Reversal             | 7         | -0.4118277142857143 | -0.13213871428571428

================================================================================
6. All rows from v_portfolio_allocation_history view
================================================================================
run_id | optimization_date | strategy_name            | allocation_weight
-------------------------------------------------------------------------------
7      | 2026-03-26        | RSI Reversal             | 1.0
8      | 2026-03-26        | Momentum                 | 0.0
24     | 2026-03-26        | Mean Reversion           | 0.0
25     | 2026-03-26        | Moving Average Crossover | 1.0
26     | 2026-03-26        | Moving Average Crossover | 1.0
27     | 2026-03-26        | Moving Average Crossover | 5.551115123125783e-17
28     | 2026-03-26        | Momentum                 | 0.0
29     | 2026-03-26        | Mean Reversion           | 0.0
30     | 2026-03-26        | RSI Reversal             | 1.0
32     | 2026-04-05        | Moving Average Crossover | 0.5
33     | 2026-04-05        | Moving Average Crossover | 0.5
36     | 2026-04-05        | RSI Reversal             | 0.0
37     | 2026-04-05        | Momentum                 | 0.0
38     | 2026-04-05        | Moving Average Crossover | 1.1102230246251565e-16
39     | 2026-04-05        | Moving Average Crossover | 1.0
```

Figure 5: Average Sharpe and drawdown per strategy — Moving Average Crossover is the only strategy with positive average Sharpe (0.326)

6.5 Backtest Execution

The Backtest tab allows users to select a strategy, enter a ticker symbol, configure the date range, set initial capital, and adjust strategy-specific parameters. On submission, the backend executes the strategy, writes results to Daily_Results and Performance_Metrics, and returns all metrics to the frontend in real time.

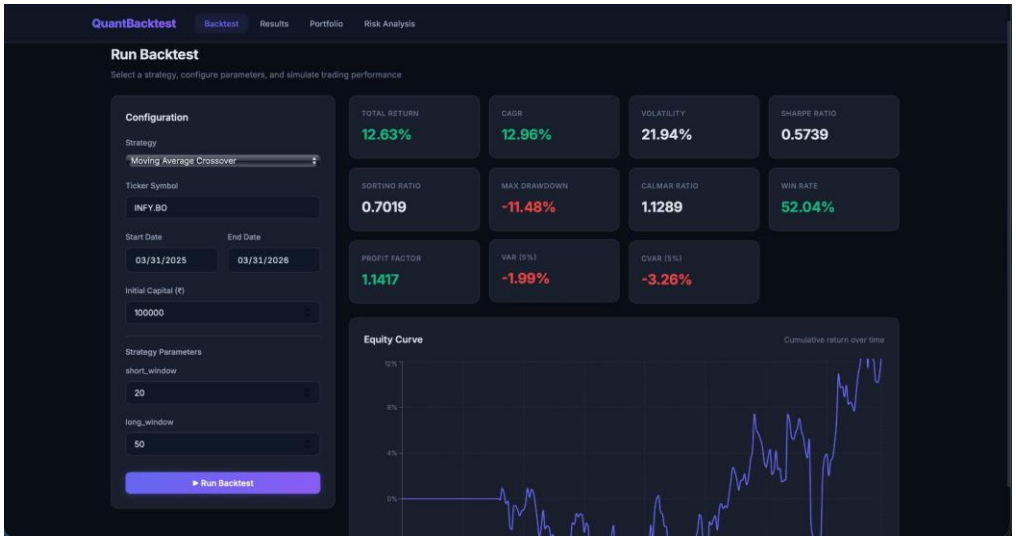


Figure 6: Backtest execution — Moving Average Crossover on INFY.BO (Mar 2025 – Mar 2026). Sharpe: 0.5739, Total Return: 12.63%, Max Drawdown: -11.48%

6.6 Backtest History

The Results tab displays all historical backtest runs retrieved from the Backtest_Runs table. Each row shows run_id, strategy name, ticker, date range, capital, and execution timestamp. The View button fetches the corresponding Performance_Metrics and Daily_Results for that run. 39 runs across RELIANCE.NS, INFY.BO, ICICIBANK.NS, HDFCBANK.NS, TCS.NS, and AAPL are stored.

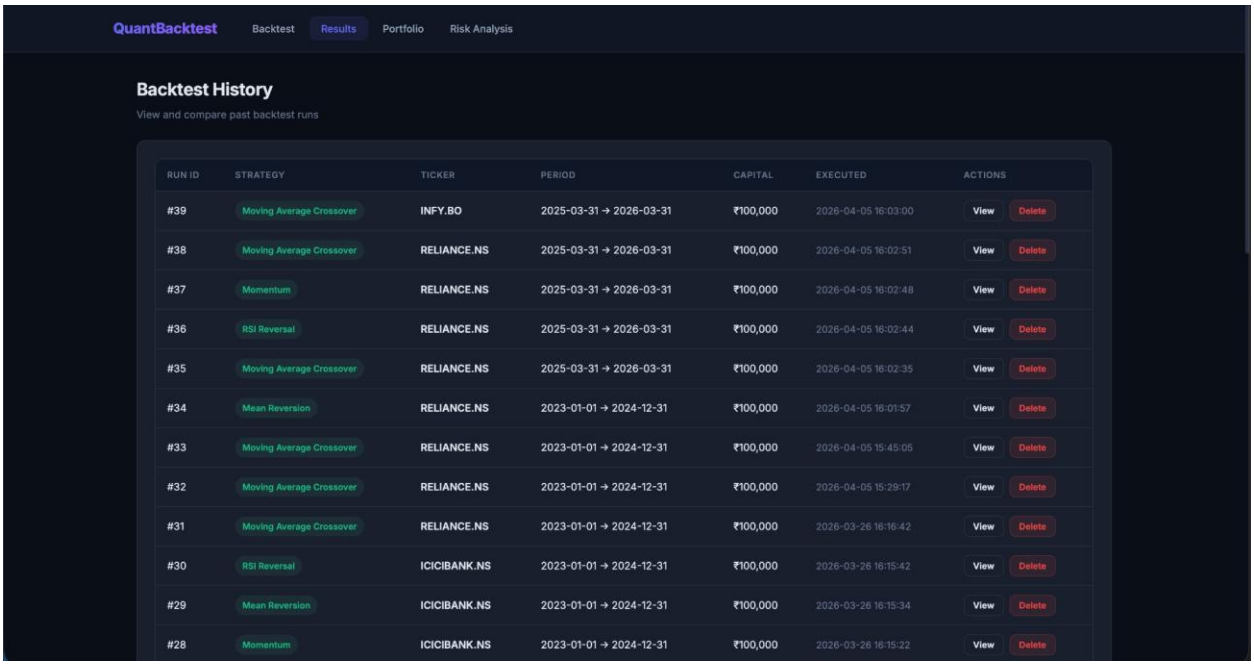


Figure 7: Backtest History — 39 runs stored across four strategies and multiple Indian equity tickers

6.7 Portfolio Optimization

The Portfolio tab allows users to select multiple backtest runs and apply mean-variance optimization. The optimizer solves for the Max Sharpe Portfolio and the Minimum Variance Portfolio using 5,000 simulated random portfolios. Optimal weights are stored in the Portfolio_Weights table and queried via the v_portfolio_allocation_history view.

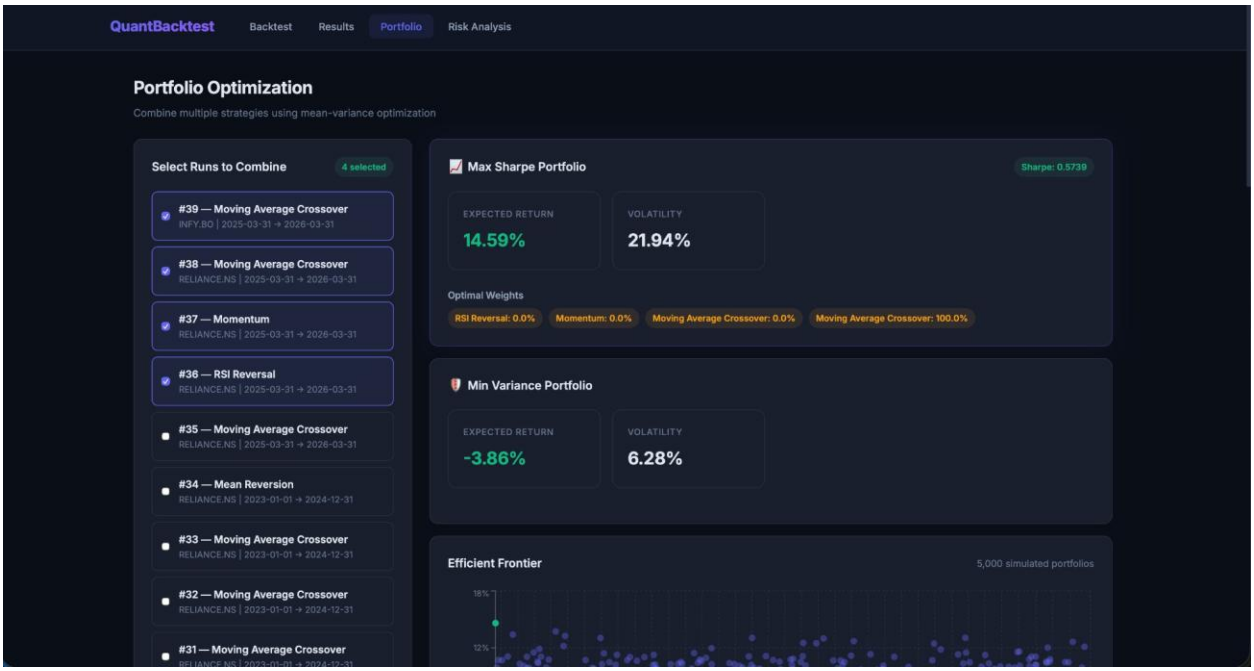


Figure 8: Portfolio Optimization — 4 runs selected. Max Sharpe portfolio (Sharpe: 0.5739) allocates 100% to Moving Average Crossover on INFY.BO

6.8 Risk Analysis Dashboard

The Risk Analysis tab provides a comprehensive risk breakdown for any selected backtest run. It computes three VaR methods, CVaR, drawdown analysis, stationarity tests, walk-forward validation, and bootstrap Sharpe significance.

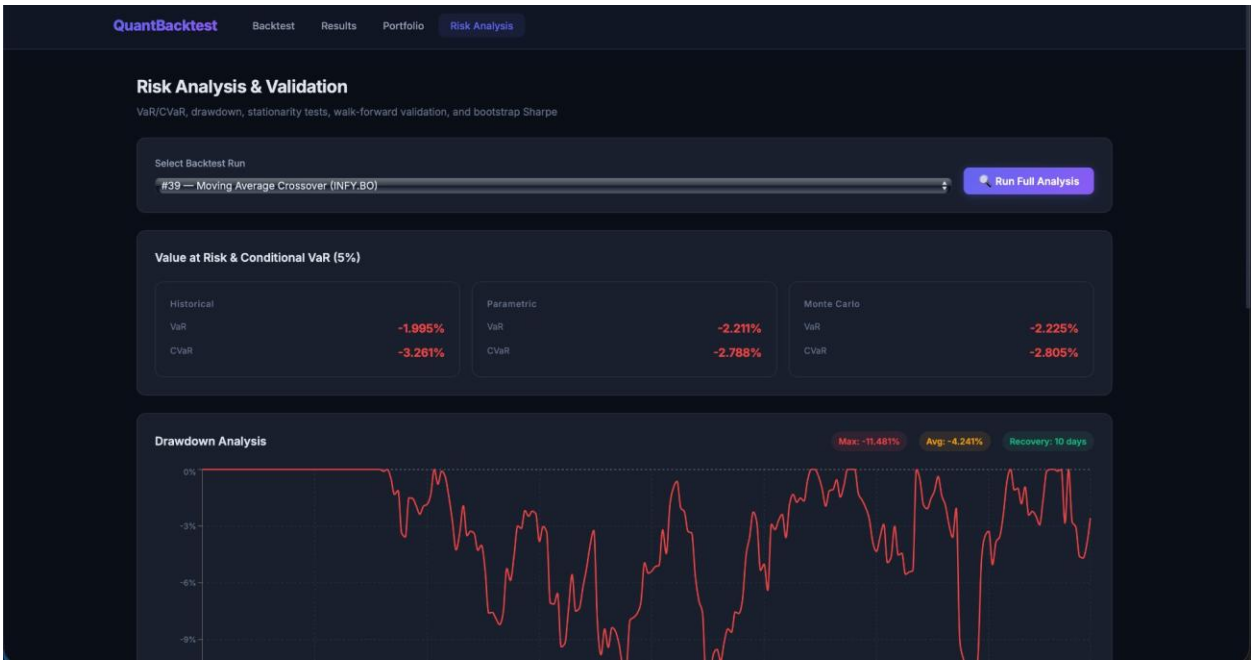


Figure 9: Risk Analysis — VaR at 5%: Historical -1.995%, Parametric -2.211%, Monte Carlo -2.225%

6.9 Drawdown Analysis

The drawdown chart plots the running peak-to-trough decline over the backtest period. The maximum drawdown of -11.481% occurred around September-October 2025, with an average drawdown of -4.241% and an average recovery time of 10 days.

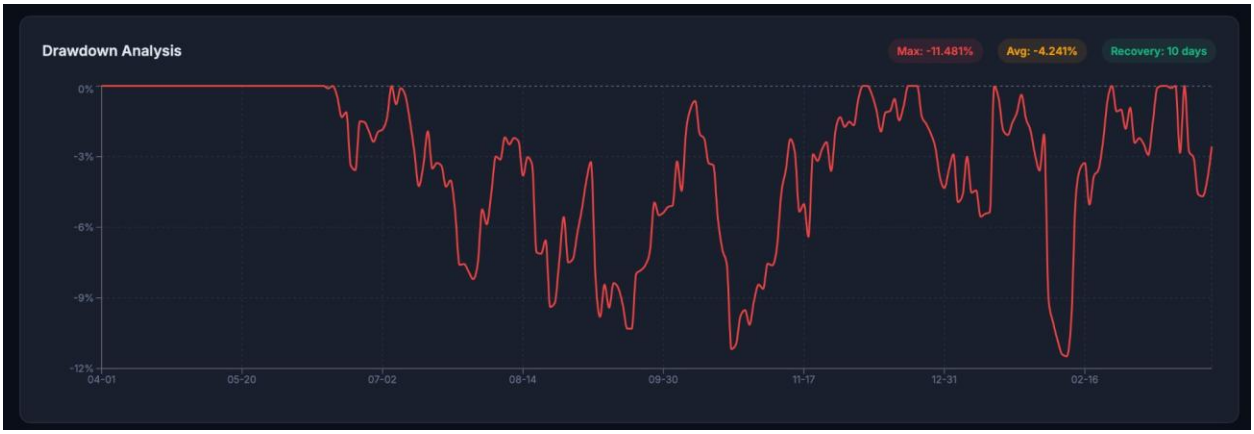


Figure 10: Drawdown Analysis — Max: -11.481%, Avg: -4.241%, Recovery: 10 days

6.10 Statistical Validation

Two stationarity tests are applied to the return series. The Augmented Dickey-Fuller (ADF) test yields a statistic of -15.55 with p-value ~0.0, strongly rejecting the unit root hypothesis. The KPSS test yields a statistic of 0.145 with p-value 0.1, failing to reject stationarity. Both tests confirm the return series is stationary, validating the use of standard statistical models.

Stationarity Tests

TEST	STATISTIC	P-VALUE	RESULT
Augmented Dickey-Fuller	-15.550491	0.000000	Stationary
KPSS	0.144550	0.100000	Stationary

Walk-Forward Validation ✓ Robust

SPLIT	IN-SAMPLE SHARPE	OUT-OF-SAMPLE SHARPE	DEGRADATION
#1	0	-1.7152	1.7152
#2	-1.2093	-0.2122	-0.9972
#3	-0.6894	2.6387	-3.328
#4	0.2029	1.5731	-1.3703
Average			-0.9951

Figure 11: Stationarity Tests (ADF + KPSS) and Walk-Forward Validation — 4 splits across the backtest period

Walk-forward validation splits the backtest period into 4 in-sample / out-of-sample pairs. The average degradation of -0.9951 indicates that out-of-sample performance generally lags in-sample performance, which is expected for trend-following strategies in noisy market conditions.

6.11 Bootstrap Sharpe Significance

Bootstrap resampling (1,000 iterations) is applied to test whether the observed Sharpe ratio of 0.6664 is statistically significant. The 95% confidence interval spans [-1.29, 2.72] with a p-value of 0.2577, meaning the result is not statistically significant at the 5% level. This is an honest assessment of strategy robustness.

Figure 12: Bootstrap Sharpe Ratio Test — p-value 0.2577, result not significant at 5% level

CHAPTER 7: CONCLUSION, LIMITATIONS & FUTURE WORK

7.1 Conclusions

This project successfully demonstrates a full-stack quantitative research platform backed by a normalized relational database. Key achievements include:

- 8-table relational schema in 3NF with complete FK constraints and CASCADE semantics
- Trigger implementation for cross-row constraint enforcement on portfolio weights
- View (v_portfolio_allocation_history) and stored procedure (sp_get_run_summary) for reusable query logic
- 9 performance indexes for efficient time-series and FK-join queries
- 39 real backtest runs stored across 4 strategies and 6 tickers
- Complete risk analytics pipeline: VaR, CVaR, drawdown, stationarity, walk-forward, bootstrap
- Mean-variance portfolio optimization with efficient frontier visualization

7.2 Limitations

- SQLite lacks native stored procedure support; the PostgreSQL schema is provided for completeness
- Market_Data table is not foreign-keyed from Backtest_Runs (ticker_symbol is a plain text column)
- All metric columns in Performance_Metrics are nullable with no completion status indicator
- Daily frequency data only; intraday backtesting not supported

7.3 Future Enhancements

- Migrate to PostgreSQL for full stored procedure and trigger support
- Add a Securities lookup table and FK-enforce ticker_symbol references
- Implement GARCH volatility modeling for more accurate risk estimation
- Add regime-switching models to account for market state changes
- Expand to multi-asset portfolio optimization across asset classes

CHAPTER 8: REFERENCES

[1] Ramakrishnan, R. & Gehrke, J. (2003). Database Management Systems (3rd ed.). McGraw-Hill.

[2] Date, C. J. (2004). An Introduction to Database Systems (8th ed.). Pearson Addison Wesley.

[3] Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). Database System Concepts (7th ed.). McGraw-Hill.

[4] Chan, E. P. (2009). Quantitative Trading: How to Build Your Own Algorithmic Trading Business. John Wiley & Sons.

[5] Pardo, R. (2008). The Evaluation and Optimization of Trading Strategies (2nd ed.). John Wiley & Sons.

[6] Markowitz, H. (1952). Portfolio Selection. The Journal of Finance, 7(1), 77–91.

[7] PostgreSQL Documentation. (2024). PL/pgSQL — SQL Procedural Language. https://www.postgresql.org/docs/current/plpgsql.html

[8] SQLite Documentation. (2024). SQLite — Features. https://www.sqlite.org/features.html

[9] Python Software Foundation. (2024). sqlite3 — DB-API 2.0 interface for SQLite databases. https://docs.python.org/3/library/sqlite3.html